

OPERATING SYSTEMS

Complete Technical Course Handbook & Detailed Syllabus Notes

Teaching Scheme: 3-0-0 (L-T-P)

Coverage: Units I through V (End-to-End)

Focus: Core Concepts, Algorithms, Architecture

UNIT I: INTRODUCTION & OPERATING SYSTEM STRUCTURES

1. Introduction to Operating Systems and Functions

An **Operating System (OS)** is system software that acts as an intermediary between the user of a computer and the computer hardware. The purpose of an OS is to provide an environment in which a user can execute programs in a convenient and efficient manner.

Core Functions of an OS:

- **Process Management:** Creation, scheduling, and termination of processes.
- **Memory Management:** Allocation and de-allocation of memory space to programs in need of resources.
- **File Management:** Organizing, storing, retrieving, naming, sharing, and protecting files.
- **Device/I/O Management:** Keeping track of all devices, deciding which process gets the device, when, and for how much time.
- **Security and Protection:** Preventing unauthorized access to programs and data.

2. Classification of Operating Systems

Operating systems have evolved over time and are classified based on their processing capabilities and user interactions:

OS Type	Description & Characteristics
Batch Systems	Users do not interact directly with the computer. Each user prepares their job on an off-line device and submits it to the computer operator. Jobs with similar needs are batched together and run through the computer as a group.
Interactive Systems	Provides direct communication between the user and the system. The user gives instructions to the OS or programs directly, receiving immediate responses.
Time Sharing Systems	A logical extension of multiprogramming. The CPU executes multiple jobs by switching among them, but the switches occur so frequently that users can interact with each program while it is running.

OS Type	Description & Characteristics
Real-Time Systems	Used when strict time requirements are placed on the operation of a processor. Contains hard real-time (strict deadlines, e.g., pacemakers) and soft real-time (missed deadlines degrade quality but don't cause failure, e.g., video streaming).
Multiprocessor Systems	Systems with two or more processors in close communication, sharing the computer bus, the clock, memory, and peripheral devices (Tightly Coupled Systems). Increases throughput and reliability.
Multiuser Systems	Allows multiple users to access a computer system at the same time (e.g., Mainframes, Linux servers).
Multithreaded Systems	Supports the execution of multiple parts of a single program (threads) concurrently, sharing the same address space.

3. Operating System Structure

The internal architecture of an operating system dictates how its components interact.

- **System Components:** Include Process Management, Memory Management, File Management, I/O System Management, Secondary-Storage Management, Networking, Protection System, and Command-Interpreter System.
- **Operating System Services:** User Interface (CLI/GUI), Program Execution, I/O Operations, File-System Manipulation, Communications, Error Detection, Resource Allocation, Accounting, and Protection.
- **Layered Structure:** The OS is broken into a number of layers (levels). The bottom layer (layer 0) is the hardware; the highest (layer N) is the user interface. Each layer uses functions and services of only lower-level layers.
- **Reentrant Kernels:** A reentrant kernel allows multiple processes to be executing in the kernel mode concurrently. It ensures that a process can be interrupted in the middle of executing kernel code, and another process can safely execute that same kernel code.

Monolithic vs. Microkernel Systems

Monolithic Architecture: The entire OS works in kernel space. All OS components (scheduler, file system, memory management) are bundled into a single massive program. (e.g., Traditional Linux/Unix). *Pros:* High performance due to low overhead of system calls. *Cons:* A bug in any component can crash the entire system.

Microkernel Architecture: Removes all non-essential components from the kernel and implements them as system and user-level programs. The kernel only provides minimal process and memory management, and a communication facility (IPC). (e.g., Mach, QNX). *Pros:* Highly reliable, secure, and extensible. *Cons:* Performance overhead due to frequent user-space/kernel-space context switching and heavy IPC.

UNIT II: CONCURRENT PROCESSES & SYNCHRONIZATION

1. Process Concept and Concurrency

A **Process** is a program in execution. It is the unit of work in a modern time-sharing system. A process includes the program code (text section), current activity (program counter, processor registers), the process stack (temporary data like function parameters, return addresses, local variables), and a data section (global variables).

Principle of Concurrency: Concurrency is the execution of multiple instruction sequences at the same time. It occurs in a multiprogramming environment where the CPU scheduler switches rapidly between processes, creating an illusion of parallel execution.

2. Inter-Process Communication (IPC) and Process Generation

Processes executing concurrently may be either independent or cooperating. Cooperating processes require an IPC mechanism.

- **Shared Memory:** A region of memory is shared by cooperating processes. Processes can exchange information by reading and writing data to the shared region. Extremely fast, but requires explicit synchronization.
- **Message Passing:** Communication takes place by means of messages exchanged between the cooperating processes via `send()` and `receive()` primitives. Better for smaller amounts of data and distributed systems.
- **Process Generation:** Using system calls like `fork()` in UNIX to create a child process. The child process is an exact duplicate of the parent process.

3. The Critical Section Problem

When concurrent processes share memory/resources, data inconsistency can occur if execution is not synchronized. The **Critical Section** is the segment of code where shared variables/files are updated.

A valid solution to the Critical Section problem must satisfy three requirements:

1. **Mutual Exclusion:** If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
2. **Progress:** If no process is executing in its critical section and some processes wish to enter, then only those processes not executing in their remainder sections can participate in deciding which will enter its critical section next.
3. **Bounded Waiting:** There must be a bound on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Software Solutions

- **Peterson's Solution:** A classic software-based solution for two processes. It uses two shared variables: `int turn;` and `boolean flag[2];`. It elegantly guarantees mutual exclusion, progress, and bounded waiting.
- **Dekker's Solution:** The first known correct software solution to the mutual exclusion problem for two processes. More complex than Peterson's, it relies strictly on shared memory for synchronization.

Hardware Solutions

- **Test and Set Operation:** A special atomic (non-interruptible) hardware instruction that reads a memory location and writes a new value into it simultaneously. Used to implement simple spinlocks.

4. Semaphores and Classical Synchronization Problems

A **Semaphore** is a synchronization tool represented as an integer variable `S`. Apart from initialization, it can only be accessed via two indivisible (atomic) atomic operations: `wait()` (or P) and `signal()` (or V).

```
wait(S) {  
    while (S <= 0); // busy wait  
    S--;  
}  
signal(S) {  
    S++;  
}
```

Classical Problems in Concurrency

- **Producer/Consumer Problem (Bounded-Buffer):** Producers create data and put it in a finite buffer; consumers remove data. Requires synchronization to ensure producers don't add to a full buffer and consumers don't remove from an empty one. Uses semaphores: `empty`, `full`, and a `mutex`.
- **Dining Philosopher Problem:** Five philosophers sit around a table with five chopsticks. A philosopher needs two chopsticks to eat. Highlights the challenges of deadlock and starvation in resource allocation.
- **Sleeping Barber Problem:** A barber shop has one barber, one barber chair, and a waiting room with n chairs. Synchronizes the actions of the barber (sleeping if no customers, working if there are) and the customers (waiting if chair available, leaving if full).

UNIT III: CPU SCHEDULING & DEADLOCK

1. CPU Scheduling Concepts

In a multiprogramming system, the objective is to have some process running at all times to maximize CPU utilization. The **CPU Scheduler (Short-term scheduler)** selects a process from the ready queue to execute and allocates the CPU to it.

- **Process States:** New, Ready, Running, Waiting (Blocked), Terminated.
- **Process Control Block (PCB):** The data structure used by the OS to store all information about a process (State, PC, CPU registers, CPU scheduling info, Memory management info, I/O status).
- **Schedulers:**
 - *Long-term Scheduler (Job Scheduler):* Selects processes from the disk and loads them into memory for execution (controls degree of multiprogramming).
 - *Short-term Scheduler (CPU Scheduler):* Selects from among the processes that are ready to execute and allocates the CPU.

◦ *Medium-term Scheduler*: Removes processes from memory (swapping) to reduce the degree of multiprogramming.

- **Threads**: A basic unit of CPU utilization. Shares code, data, and OS resources with other threads within the same process, but has its own program counter, register set, and stack.

Scheduling Performance Criteria

Turnaround Time (TAT) = Completion Time - Arrival Time

Waiting Time (WT) = Turnaround Time - Burst Time

Goals: Maximize CPU Utilization and Throughput. Minimize Turnaround Time, Waiting Time, and Response Time.

Scheduling Algorithms

Algorithm	Description & Characteristics
First-Come, First-Served (FCFS)	Non-preemptive. The process that requests the CPU first is allocated the CPU first. Suffers from the <i>Convoy Effect</i> (short processes get stuck waiting behind long processes).
Shortest-Job-First (SJF)	Non-preemptive or Preemptive (SRTF - Shortest Remaining Time First). Allocates the CPU to the process with the smallest next CPU burst. Gives the optimal minimum average waiting time. Difficult to predict future burst lengths.
Priority Scheduling	A priority number is associated with each process, and the CPU is allocated to the process with the highest priority. Can lead to <i>Starvation</i> (low priority processes never execute), which is solved by <i>Aging</i> (gradually increasing priority over time).
Round Robin (RR)	Preemptive. Designed for time-sharing systems. Each process gets a small unit of CPU time (time quantum). If the burst > quantum, the process is preempted and put at the back of the ready queue.

Multiprocessor Scheduling: More complex due to load balancing and CPU affinity (a process has an affinity for the processor on which it is currently running to preserve cache warmth).

2. Deadlock

A **Deadlock** is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.

Deadlock Characterization (Coffman Conditions)

A deadlock can arise if and only if all four conditions hold simultaneously in a system:

1. **Mutual Exclusion**: At least one resource must be held in a non-sharable mode.
2. **Hold and Wait**: A process must be holding at least one resource and waiting to acquire additional resources held by other processes.
3. **No Preemption**: Resources cannot be preempted; a resource can be released only voluntarily by the process holding it.

4. **Circular Wait:** A set of waiting processes $\{P_0, P_1, \dots, P_n\}$ must exist such that P_0 is waiting for a resource held by P_1 , P_1 is waiting for P_2 , ..., and P_n is waiting for P_0 .

Handling Deadlocks

- **Deadlock Prevention:** Ensure that at least one of the four Coffman conditions cannot hold. (e.g., forcing processes to request all resources at once prevents Hold and Wait).
- **Deadlock Avoidance:** Requires the OS to be given in advance additional information concerning which resources a process will request and use during its lifetime.
Banker's Algorithm: A classic avoidance algorithm that simulates the allocation for predetermined maximum possible amounts of all resources, and then makes an "s-state" check to test for possible deadlock conditions before permanently deciding on an allocation.
- **Deadlock Detection and Recovery:** Allow the system to enter a deadlocked state, run an algorithm to detect it, and then recover (e.g., by aborting one or more processes or preempting resources).

UNIT IV: MEMORY MANAGEMENT

1. Background and Early Systems

Memory management is the functionality of an OS which handles or manages primary memory and moves processes back and forth between main memory and disk during execution.

- **Bare Machine:** Early computers had no OS. The user had complete control over the entire memory space.
- **Resident Monitor:** The first rudimentary OS. It resided in a small portion of memory and simply transferred control to a program, regaining control once the program terminated.
- **Multiprogramming with Fixed Partitions (MFT):** Memory is divided into fixed-sized partitions. Each partition holds exactly one process. Suffers from severe *Internal Fragmentation*.
- **Multiprogramming with Variable Partitions (MVT):** The OS keeps a table indicating which parts of memory are available and which are occupied. When a process arrives, it is allocated memory exactly fitting its size. Over time, memory becomes fragmented (*External Fragmentation*). Addressed via Compaction.

2. Non-Contiguous Allocation: Paging and Segmentation

Paging

A memory management scheme that eliminates the need for contiguous allocation of physical memory.

- Physical memory is broken into fixed-sized blocks called **Frames**.
- Logical memory is broken into blocks of the same size called **Pages**.
- The OS uses a **Page Table** to map logical pages to physical frames.
- *Advantage:* Eliminates external fragmentation. *Disadvantage:* Introduces small internal fragmentation at the last page of a process. Requires extra memory for page tables.

Segmentation

A memory-management scheme that supports the user view of memory. A logical address space is a collection of **Segments** (e.g., main program, function, local variables, stack). Each segment has a name and length.

Paged Segmentation: Combines both models. Segments are divided into pages. The segment table contains pointers to page tables for each segment.

3. Virtual Memory and Demand Paging

Virtual Memory is a technique that allows the execution of processes that are not completely in memory. It provides the illusion of an extremely large main memory.

- **Demand Paging:** Pages are only loaded into memory when they are demanded during execution. If a process tries to access a page not in memory, a **Page Fault** occurs. The OS traps the interrupt, brings the required page from the disk into a free frame, updates the page table, and restarts the instruction.
- **Performance of Demand Paging:** The Effective Access Time (EAT) depends heavily on the page fault rate (\$p\$).

$$EAT = (1 - p) \times \text{Memory Access Time} + p \times \text{Page Fault Overhead}$$

Page Replacement Algorithms

When a page fault occurs and memory is full, the OS must swap out an existing page. The goal is to minimize the page fault rate.

Algorithm	Description
FIFO (First-In, First-Out)	Replaces the oldest page in memory. Simple to implement but suffers from <i>Belady's Anomaly</i> (increasing the number of page frames can increase the number of page faults).
Optimal Page Replacement	Replaces the page that will not be used for the longest period of time. Yields the lowest page-fault rate but is practically impossible to implement since it requires future knowledge. Used as a benchmark.
LRU (Least Recently Used)	Replaces the page that has not been used for the longest period of time. An excellent approximation of the Optimal algorithm, implemented using counters or a stack.

Thrashing & Locality of Reference

Thrashing: A severe performance degradation that occurs when a system is spending more time paging (swapping pages in and out of the disk) than executing. It happens when the degree of multiprogramming is too high, and processes do not have enough frames to hold their active working sets.

Locality of Reference: A phenomenon where a process accesses a relatively small portion of its address space during a particular phase of execution. *Spatial Locality:* Elements close to each other are accessed together. *Temporal Locality:* Recently accessed elements will be accessed again soon. Demand paging works effectively *because* of the locality model.

UNIT V: I/O MANAGEMENT, DISK SCHEDULING, AND FILE SYSTEMS

1. I/O Management and Disk Storage

The OS manages the communication between peripheral devices (I/O devices) and the processor.

- **I/O Subsystems:** Hides the peculiarities of specific hardware devices from the user. Includes a memory-management component (buffering, caching, spooling), general device-driver interface, and drivers for specific hardware devices.
- **I/O Buffering:** A buffer is a memory area that stores data being transferred between two devices or between a device and an application. It helps cope with speed mismatches, data transfer size mismatches, and maintains copy semantics.

Disk Storage and Scheduling

Magnetic disks provide the bulk of secondary storage. The OS schedules disk I/O requests to minimize seek time (the time for the disk arm to move the heads to the cylinder containing the desired sector).

Disk Scheduling Algorithm	Mechanism and Properties
FCFS (First-Come, First-Served)	Serves requests in the order they arrive. Fair, but does not provide the fastest service.
SSTF (Shortest Seek Time First)	Selects the request with the minimum seek time from the current head position. May cause starvation for requests far from the head.
SCAN (Elevator Algorithm)	The disk arm starts at one end of the disk and moves toward the other end, servicing requests as it goes, then reverses direction.
C-SCAN (Circular SCAN)	Like SCAN, but when it reaches the end, it immediately returns to the beginning without servicing any requests on the return trip. Provides a more uniform wait time.
LOOK / C-LOOK	Variants of SCAN/C-SCAN. The arm only goes as far as the final request in each direction, rather than going all the way to the physical end of the disk.

RAID (Redundant Array of Independent Disks)

A technology that combines multiple physical disk drive components into one or more logical units for the purposes of data redundancy, performance improvement, or both.

- **RAID 0:** Block-level striping without parity. High performance, no redundancy.
- **RAID 1:** Mirroring. Excellent redundancy, but 50% capacity penalty.
- **RAID 5:** Block-level striping with distributed parity. Good balance of performance and fault tolerance (can survive 1 drive failure).
- **RAID 6:** Block-level striping with double distributed parity. Can survive 2 drive failures.

2. File Systems

A **File** is a logical collection of contiguous related information recorded on secondary storage.

- **File Organization and Access Mechanisms:**

- *Sequential Access*: Information is processed in order, one record after the other (like magnetic tape).
- *Direct (Relative) Access*: Programs can read and write records rapidly in no particular order. Allows immediate access to any block.

- **File Directories**: A directory is a symbol table that translates file names into their directory entries. Types include Single-Level, Two-Level, and Tree-Structured (the standard in modern OS).

- **File Sharing**: Operating systems allow users to share files via mechanisms like hard links and soft (symbolic) links.

Implementation Issues and Protection

- **Allocation Methods**: How disk blocks are allocated for files.

- *Contiguous Allocation*: Each file occupies a set of contiguous blocks on the disk. Fast, but suffers from external fragmentation.
- *Linked Allocation*: Each file is a linked list of disk blocks. No external fragmentation, but does not support direct access efficiently.
- *Indexed Allocation*: Brings all pointers together into one location called the *Index Block*. Solves external fragmentation and supports direct access (used by UNIX inodes).

- **File Protection and Security**: Ensuring that files are safe from physical damage (reliability/backups) and improper access (protection). Access Control Lists (ACL) map users/groups to Read, Write, and Execute (RWX) permissions.